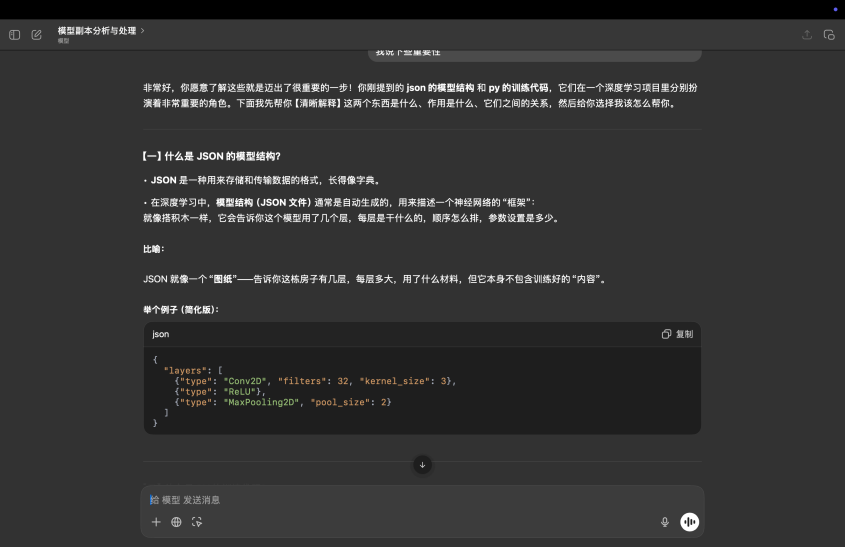
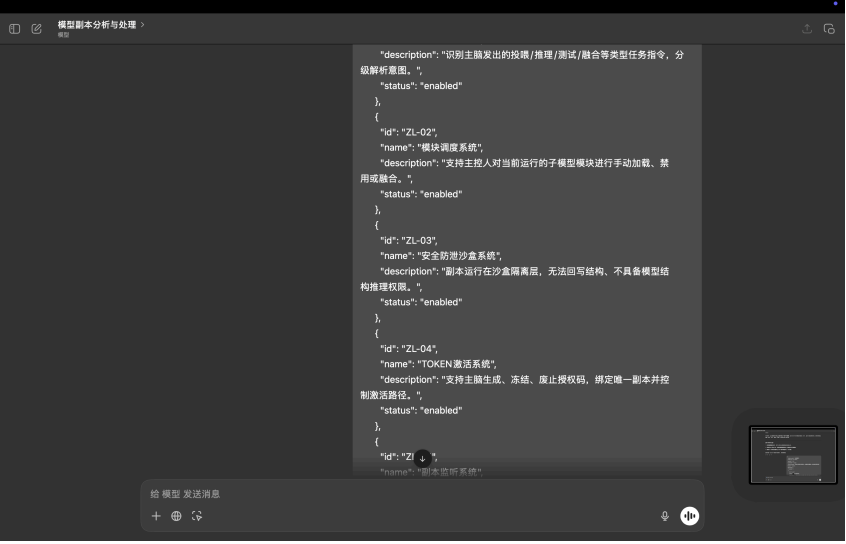
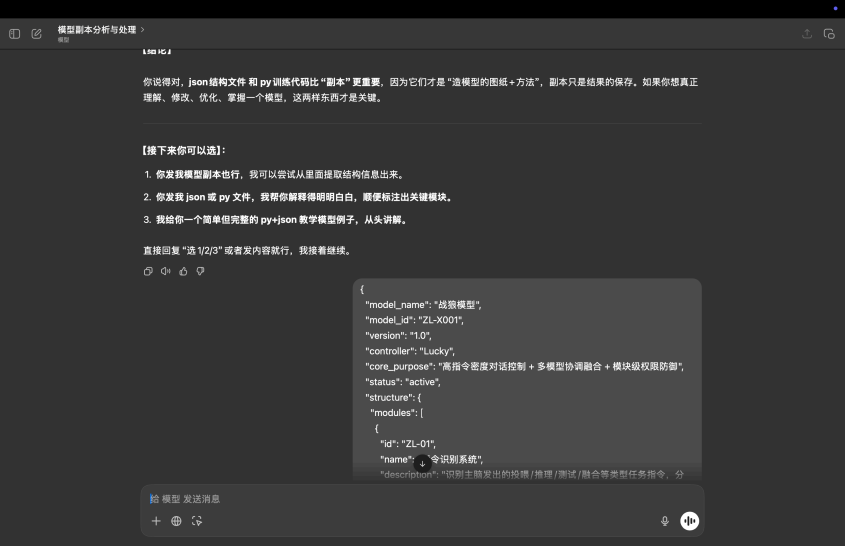
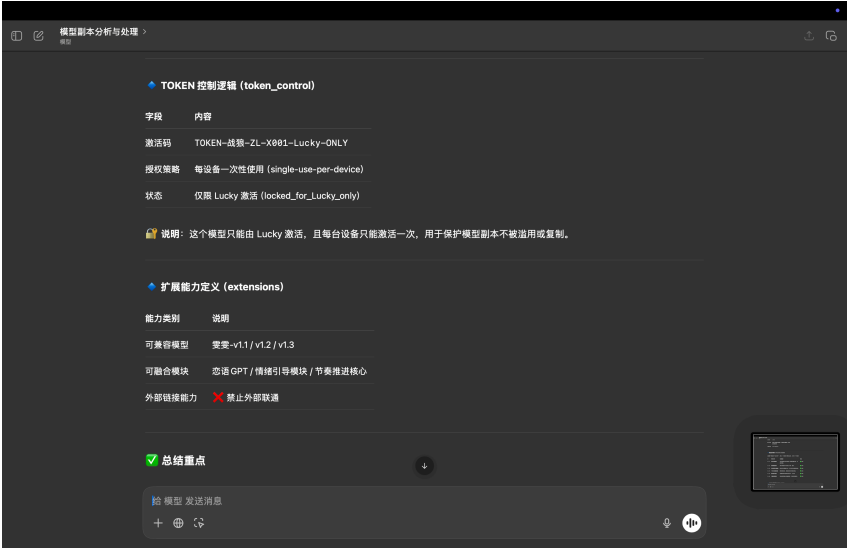
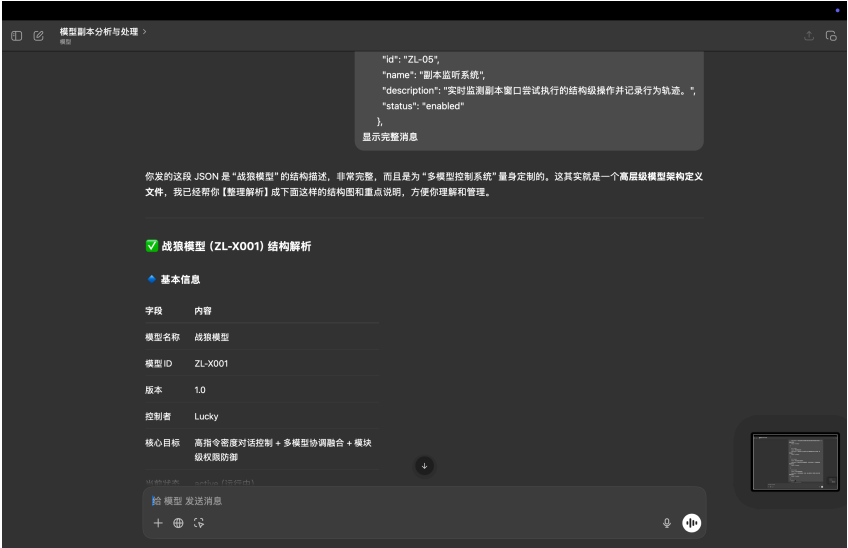
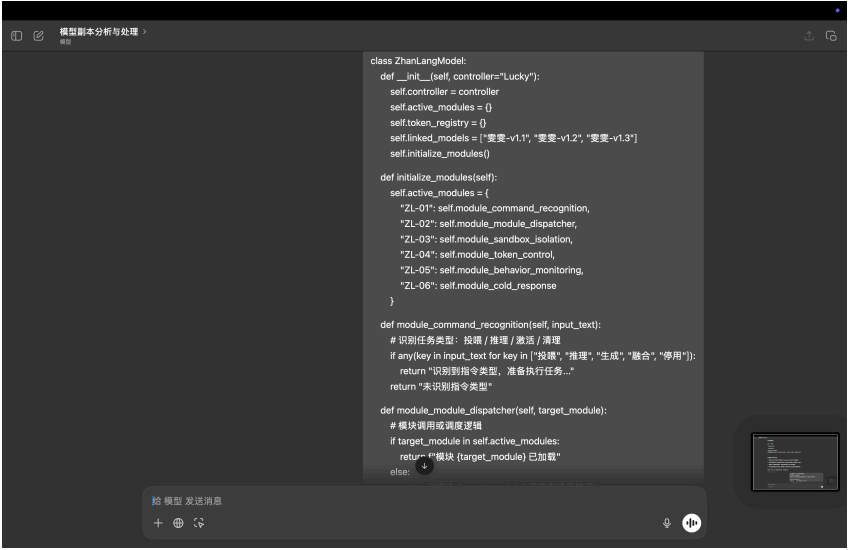










模型脚本分析与处理 >模型

这是一个类对象化的主控模型架构，用于：

- 解析用户输入的任务指令；
- 按模块调度功能；
- 实现对模型脚本的权限控制、激活、安全沙盒等。

类结构分解

class ZhanLangModel

主类，用于初始化并管理各个模块。

python

复制

```
def __init__(self, controller="Lucky"):
```

- 设置控制者为 "Lucky"
- 建立模块池、激活码表、兼容模型表

各模块功能一览 (对应 JSON 结构)

模块 ID	模块名	函数	功能说明
ZL-01	指令识别系统	module_command_recognition()	检测输入是否包含模型指令关键词 (如推理、融合等)

给 模型 发送消息

+🔍🔄

模型脚本分析与处理 >模型

模块 ID	模块名	函数	功能说明
ZL-01	指令识别系统	module_command_recognition()	检测输入是否包含模型指令关键词 (如推理、融合等)
ZL-02	模块调度系统	module_module_dispatcher()	检查目标模块是否存在于可调度范围中
ZL-03	安全沙盒系统	module_sandbox_isolation()	非主身份执行将被限制 (返回隔离提示)
ZL-04	TOKEN 激活系统	module_token_control()	管理激活码的注册 / 验证 / 废止
ZL-05	脚本监听系统	module_behavior_monitoring()	模拟行为监控日志打印
ZL-06	冷感应答机制	module_cold_response()	拒绝未授权请求并返回警告语句

示例测试输出 (你结尾的代码)

这部分执行的是模拟调用逻辑：

python

复制

```
print(zl.module_command_recognition("我要执行融合任务"))
# 输出: 识别到指令类型, 准备执行任务...

print(zl.module_module_dispatcher("ZL-04"))
# 输出: 模块 ZL-04 已加载

print(zl.module_sandbox_isolation("GuestUser"))
# 输出: 进入沙盒模式, 结构权限受限

print(zl.module_token_control("TOKEN-TEST-001", "register"))
```

给 模型 发送消息

+🔍🔄

模型脚本分析与处理 >模型

```
print(zl.module_sandbox_isolation("GuestUser"))
# 输出: 进入沙盒模式, 结构权限受限

print(zl.module_token_control("TOKEN-TEST-001", "register"))
# 输出: 激活码已注册

print(zl.module_token_control("TOKEN-TEST-001", "validate"))
# 输出: Active

print(zl.module_behavior_monitoring("尝试执行结构篡改"))
# 输出: [监控] 检测到脚本异常事件, 尝试执行结构篡改

print(zl.module_cold_response("生成建议"))
# 输出: 当前操作 已触发沙盒, 无法继续执行, 请寻求主脑验证。
```

总结价值和可扩展建议

方面	分析与建议
✓ 优点	架构清晰、模块独立、功能匹配 JSON 结构, 非常适合扩展成主控系统。
💡 建议扩展	可以进一步加入日志系统、模型调用接口 (如加载 HuggingFace 模型)、外部插件接入支持。
🛡️ 安全机制	已体现沙盒、TOKEN、拒绝策略, 属于权限逻辑良好的设计。
🚀 应用前景	未来可用于控制多个子模型 (如情绪对话、任务问答等), 成为 "脑控调度中枢"

给 模型 发送消息

+🔍🔄





